LAB ASSIGNMENT-1

PART A – COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

1. STUDENT

STUDENT (USN, SName, Address, Phone, Gender)

- **Description:** Stores personal details of students.
- **Primary Key:** USN
- **Attributes:**
  - USN – Unique Student Number
  - SName – Student Name
  - Address – Residential Address
  - Phone – Contact Number
  - Gender – Gender of the student
- **Purpose:** Uniquely identifies each student in the college database.

2. SEMSEC

SEMSEC (SSID, Sem, Sec)

- **Description:** Represents semester and section details.
- **Primary Key:** SSID
- **Attributes:**
  - SSID – Semester-Section ID
  - Sem – Semester number
  - Sec – Section (A, B, C, etc.)
- **Purpose:** Helps group students based on semester and section.

3. CLASS

CLASS (USN, SSID)

- **Description:** Associates students with their semester and section.
- **Primary Key:** (USN, SSID) *(Composite Key)*
- **Foreign Keys:**
  - USN → references STUDENT(USN)
  - SSID → references SEMSEC(SSID)
- **Purpose:** Resolves the relationship between STUDENT and SEMSEC.

4. SUBJECT

SUBJECT (Subcode, Title, Sem, Credits)

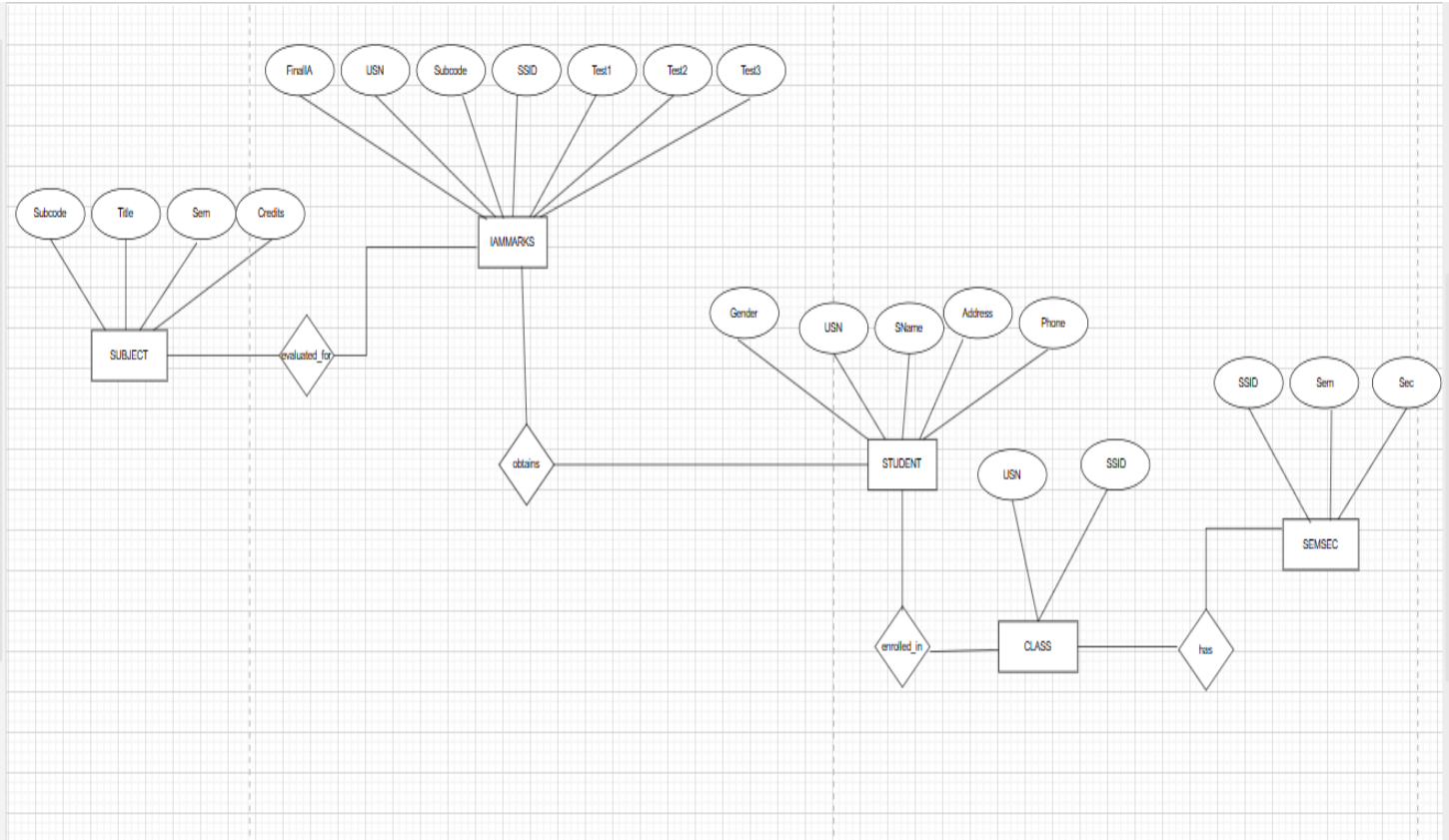
- **Description:** Stores subject/course information.
- **Primary Key:** Subcode

- **Attributes:**
  - Subcode – Unique subject code
  - Title – Subject name
  - Sem – Semester in which subject is offered
  - Credits – Number of credits
- **Purpose:** Defines academic subjects offered by the college.

5. IAMARKS

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

- **Description:** Stores internal assessment marks of students.
- **Primary Key:** (USN, Subcode, SSID) *(Composite Key)*
- **Foreign Keys:**
  - USN → references STUDENT(USN)
  - Subcode → references SUBJECT(Subcode)
  - SSID → references SEMSEC(SSID)
- **Attributes:**
  - Test1, Test2, Test3 – Internal test marks
  - FinalIA – Final internal assessment score
- **Purpose:** Tracks student performance in each subject.



PART B – COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Sex, Salary, Superssn, Dno)

DEPARTMENT (Dno, Dname, Mgrssn, Mgrstartdate)

DLOCATION (Dno, Dloc)

PROJECT (PNo, PName, PLocation, DNo)

WORKS_ON (SSN, PNo, Hours)

1. STUDENT

STUDENT (USN, SName, Address, Phone, Gender)

- **Description:** Stores personal details of students.

- **Primary Key:** USN
- **Attributes:**
  - SName – Student name
  - Address – Residential address
  - Phone – Contact number
  - Gender – Gender of the student
- **Purpose:** Uniquely identifies each student.

2. SEMSEC

SEMSEC (SSID, Sem, Sec)

- **Description:** Represents semester and section details.
- **Primary Key:** SSID
- **Attributes:**
  - Sem – Semester number
  - Sec – Section (A, B, etc.)
- **Purpose:** Groups students based on semester and section.

3. CLASS

CLASS (USN, SSID)

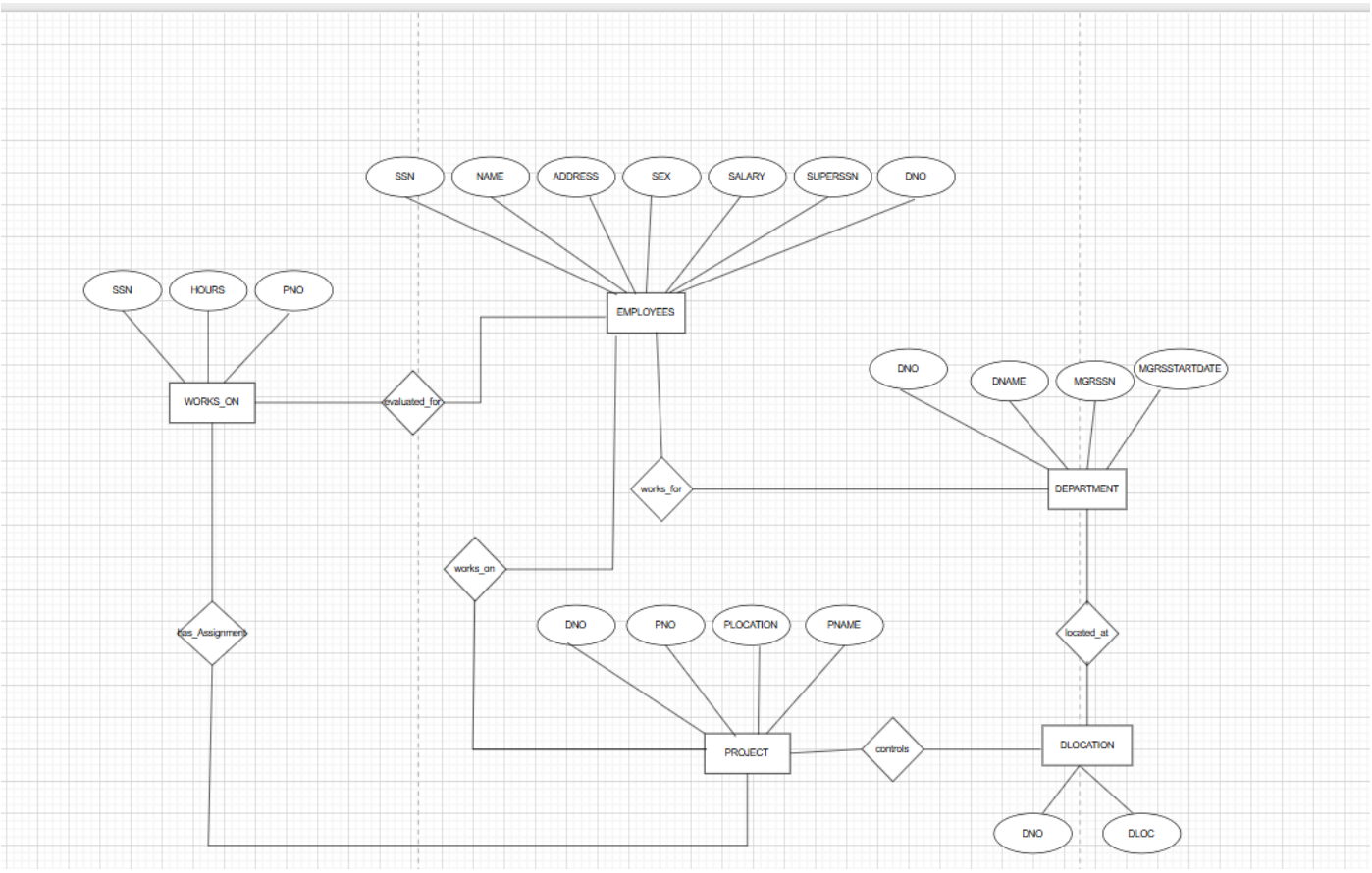
- **Description:** Associates students with their semester and section.
- **Primary Key:** *(USN, SSID) (Composite Key)*
- **Foreign Keys:**
  - USN → references **STUDENT(USN)**
  - SSID → references **SEMSEC(SSID)**

- **Purpose:** Resolves the many-to-many relationship between STUDENT and SEMSEC.

4. SUBJECT

SUBJECT (Subcode, Title, Sem, Credits)

- **Description:** Stores subject/course information.
- **Primary Key:** Subcode
- **Attributes:**
  - Title – Subject name
  - Sem – Semester in which subject is offered
  - Credits – Number of credits
- **Purpose:** Defines academic subjects offered by the college.



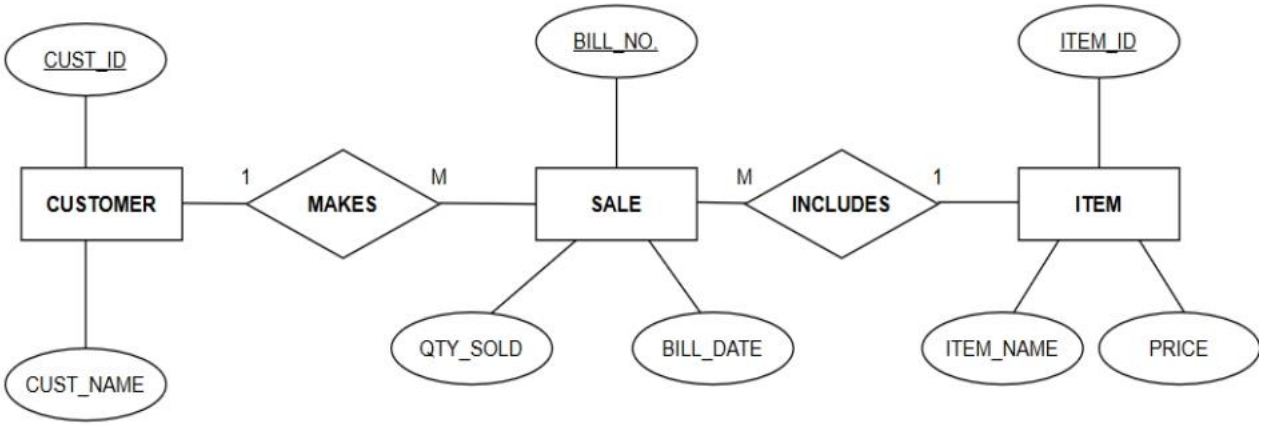
PART C – CUSTOMER SALE SCHEMA

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name:string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)



1. CUSTOMER

CUSTOMER (Cust_id, Cust_name)

- Description: Stores details of customers.
- Primary Key: Cust_id
- Attributes:
    - Cust_name – Name of the customer
- Purpose: Identifies customers who make purchases.

2. ITEM

ITEM (Item_id, Item_name, Price)

- Description: Stores information about items available for sale.
- Primary Key: Item_id
- Attributes:
  - Item_name – Name of the item
  - Price – Price of the item
- Purpose: Maintains item details and pricing information.

3. SALE

SALE (Bill_no, Bill_date, Qty_sold)

- Description: Stores sales transaction details.
- Primary Key: Bill_no
- Attributes:
  - Bill_date – Date of the sale
  - Qty_sold – Quantity of items sold
- Purpose: Records individual sales transactions.

1. Create all relations with PRIMARY KEY and FOREIGN KEY constraints

STUDENT

```
CREATE TABLE STUDENT (
   USN VARCHAR(10) PRIMARY KEY,
   SName VARCHAR(50),
   Address VARCHAR(100),
   Phone VARCHAR(15),
   Gender CHAR(1)
);
```

SEMSEC

```
CREATE TABLE SEMSEC (
   SSID INT PRIMARY KEY,
   Sem INT,
   Sec CHAR(1)
);
```

CLASS

```
CREATE TABLE CLASS (
   USN VARCHAR(10),
   SSID INT,
   PRIMARY KEY (USN, SSID),
   FOREIGN KEY (USN) REFERENCES STUDENT(USN),
   FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

SUBJECT

```
CREATE TABLE SUBJECT (
   Subcode VARCHAR(10) PRIMARY KEY,
   Title VARCHAR(50),
   Sem INT,
   Credits INT
);
```

IAMARKS

```
CREATE TABLE IAMARKS (
   USN VARCHAR(10),
   Subcode VARCHAR(10),
   SSID INT,
   Test1 INT,
   Test2 INT,
   Test3 INT,
   FinalIA INT,
   PRIMARY KEY (USN, Subcode, SSID),
   FOREIGN KEY (USN) REFERENCES STUDENT(USN),
   FOREIGN KEY (Subcode) REFERENCES SUBJECT(Subcode),
   FOREIGN KEY (SSID) REFERENCES SEMSEC(SSID)
);
```

CUSTOMER

```
CREATE TABLE CUSTOMER (
   Cust_id INT PRIMARY KEY,
   Cust_name VARCHAR(50)
);
```

ITEM

```
CREATE TABLE ITEM (
   Item_id INT PRIMARY KEY,
   Item_name VARCHAR(50),
   Price INT
);
```

SALE

```
CREATE TABLE SALE (
   Bill_no INT PRIMARY KEY,
   Bill_date DATE,
   Cust_id INT,
   Item_id INT,
   Qty_sold INT,
   FOREIGN KEY (Cust_id) REFERENCES CUSTOMER(Cust_id),
   FOREIGN KEY (Item_id) REFERENCES ITEM(Item_id)
);
```

2.Add a new column Email to STUDENT

```
ALTER TABLE STUDENT
ADD Email VARCHAR(50);
```

3.Apply NOT NULL constraint to Salary and Price

EMPLOYEE (Salary)

```
ALTER TABLE EMPLOYEE
MODIFY Salary INT NOT NULL;
```

ITEM (Price)

ALTER TABLE ITEM
MODIFY Price INT NOT NULL;

4.Rename attribute SName to StudentName in STUDENT

ALTER TABLE STUDENT
RENAME COLUMN SName TO StudentName;

5.Drop the DLOCATION relation

DROP TABLE DLOCATION;

DML QUESTIONS (Data Manipulation Language)

1.Insert at least 10 tuples

CUSTOMER

```
INSERT INTO CUSTOMER VALUES
(1, 'Amit'),
(2, 'Neha'),
(3, 'Rahul'),
(4, 'Sneha'),
(5, 'Karan'),
(6, 'Priya'),
(7, 'Rohit'),
(8, 'Anjali'),
(9, 'Vikas'),
(10, 'Pooja');
```

OUTPUT:

Cust_id	Cust_name
1	Amit
2	Neha
3	Rahul
4	Sneha
5	Karan

ITEM

```
INSERT INTO ITEM VALUES
(101, 'Pen', 50),
(102, 'Notebook', 120),
(103, 'Bag', 850),
(104, 'Calculator', 600),
(105, 'Bottle', 250),
(106, 'Shoes', 1200),
(107, 'Mouse', 700),
(108, 'Keyboard', 900),
(109, 'Charger', 450),
(110, 'Headphones', 1500);
```

OUTPUT:

Item_id	Item_name	Price
101	Pen	50
102	Notebook	120
103	Bag	850
104	Calculator	600
105	Bottle	250

SALE

```
INSERT INTO SALE VALUES
(1001, CURDATE(), 1, 101, 5),
(1002, CURDATE(), 2, 103, 1),
(1003, CURDATE(), 3, 105, 2),
(1004, CURDATE(), 4, 106, 1),
(1005, CURDATE(), 5, 104, 1),
(1006, CURDATE(), 5, 110, 1),
(1007, CURDATE(), 6, 102, 3),
(1008, CURDATE(), 7, 109, 2),
(1009, CURDATE(), 8, 107, 1),
(1010, CURDATE(), 9, 108, 1);
```

OUTPUT:

Bill_no	Bill_date	Cust_id	Item_id	Qty_sold
1001	2026-03-01	1	101	5
1002	2026-03-01	2	103	1
1003	2026-03-01	3	105	2
1004	2026-03-01	4	106	1
1005	2026-03-01	5	104	1

2.List all bills of the current date with customer and item names

```
SELECT s.bill_no, s.bill_date, c.cust_name, i.item_name
FROM sale s
JOIN customer c ON s.cust_id = c.cust_id
JOIN item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE();
```

OUTPUT:

bill_no	bill_date	cust_name	item_name
1001	2026-03-01	Amit	Pen
1002	2026-03-01	Neha	Bag
1003	2026-03-01	Rahul	Bottle
1004	2026-03-01	Sneha	Shoes
1005	2026-03-01	Karan	Calculator

3.Display total bill amount (qty × price)

```
SELECT s.bill_no,
       c.cust_name,
       i.item_name,
       s.qty_sold,
       i.price,
       (s.qty_sold * i.price) AS total_amount
FROM sale s
JOIN customer c ON s.cust_id = c.cust_id
JOIN item i ON s.item_id = i.item_id;
```

OUTPUT:

bill_no	cust_name	item_name	qty_sold	price	total_amount
1001	Amit	Pen	5	50	250
1002	Neha	Bag	1	850	850
1003	Rahul	Bottle	2	250	500
1004	Sneha	Shoes	1	1200	1200
1005	Karan	Calculator	1	600	600

4.List customers who purchased products priced above 200

```
SELECT DISTINCT c.cust_name
FROM customer c
JOIN sale s ON c.cust_id = s.cust_id
JOIN item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

OUTPUT:

	cust_name
▶	Neha
	Karan
	Rahul
	Sneha
	Anjali

5. Count number of products purchased by each customer

```
SELECT c.cust_name, COUNT(s.item_id) AS total_products
FROM customer c
JOIN sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_name;
```

OUTPUT:

	cust_name	total_products
▶	Amit	1
	Neha	1
	Rahul	1
	Sneha	1
	Karan	2

6. List products purchased by customer with Cust_id = 5

```
SELECT i.item_name
FROM sale s
JOIN item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

OUTPUT:

	item_name
▶	Calculator
	Headphones

7.Create a VIEW showing bill details with amount

```
CREATE VIEW bill_view AS
SELECT s.bill_no,
       s.bill_date,
       s.cust_id,
       s.item_id,
       i.price,
       s.qty_sold,
       (s.qty_sold * i.price) AS amount
FROM sale s
JOIN item i ON s.item_id = i.item_id;
```

OUTPUT:

	bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
▶	1001	2026-03-01	1	101	50	5	250
	1002	2026-03-01	2	103	850	1	850
	1003	2026-03-01	3	105	250	2	500
	1004	2026-03-01	4	106	1200	1	1200
	1005	2026-03-01	5	104	600	1	600

8.Create a VIEW showing daily sales for the last one week

```sql
CREATE VIEW weekly_sales AS
SELECT s.bill_date,
       SUM(s.qty_sold * i.price) AS total_sales
FROM sale s
JOIN item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
```

OUTPUT:

	bill_date	total_sales
▶	2026-03-01	7760